

深度学习大作业：AI 文本检测

明裕佳 2023111834

2026 年 6 月 29 日

1 介绍

1.1 问题背景

随着大语言模型的快速发展，机器生成文本在流畅性、逻辑性以及语言质量上已经接近甚至在某些场景下达到人类水平，我们越来越难以区分文本是有人类撰写还是由模型生成的。也正是因为大语言模型能够快速生成文本、以及与人类生成的文本越来越像，人们开始在更广泛的领域更高频率地使用大语言模型（后称 LLM）生成文本。这降低了人类的生产成本，但同时也带来了社会风险。例如，LLM 可能在新闻传播中生成虚假信息，加剧谣言传播；LLM 可能被学生用来代写论文，导致学术诚信的问题等。此外，如果 AI 生成文本被进一步用于训练下一代模型，还可能导致数据质量下降与模型性能退化。因此，判断给定文本是人类生成的，还是 LLM 生成的，或者更进一步，是哪种 LLM 生成的，成为当前亟需解决的重要问题。

针对 LLM 生成文本检测问题，已有研究将其形式化为一个二分类任务：判断输入文本是人类撰写还是由模型生成。围绕这个任务，现有方法通常分为统计与传统机器学习方法、神经网络方法、零样本与概率统计方法和水印方法（Wu et al., 2024）。第一，统计与传统机器学习方法（Beresneva, 2016; Jawahar et al., 2020）。早期主要依赖词频、句法结构、困惑度等统计特征进行分类，但在面对 GPT-3.5 及以上模型时性能明显下降（Uchendu et al., 2021）。第二，神经网络方法（Li et al., 2023; Liu et al., 2023）。这些文献通常使用 BERT 等预训练模型进行微调，通过学习人类文本与机器文本的分布差异进行判别。这类方法在同分布数据上表现较好，但对跨模型或跨领域数据泛化能力较弱，关于跨领域数据我们在下一段阐述。第三，零样本与概率统计方法。这类方法不依赖大量标注数据，而是利用模型生成文本在概率空间中的“局部曲率异常”或困惑度差异来进行检测，在跨模型场景中具有较强的鲁棒性，代表性方法包括 DetectGPT 和 Fast-DetectGPT（Mitchell et al., 2023; Bao et al., 2023）。这也是近年来最热门的方法。第四，水印方法。通过在生成过程中嵌入隐式标记，使得后续检测可以通过统计 token 分布或触发模式来识别是否为模型生成文本，虽然这种方法更多适用于工程上如何解决区分的问题，对于我们直接拿到数据进行训练、判断的任务不太适用，但本文认为思想仍然可以借鉴。

在现有的数据集方面，可以通过“文本由哪类模型生成”和“文本属于什么应用场景”来分。在模型类型上，近年来的数据集主要使用 GPT-3.5、ChatGPT、GPT-4 以及 LLaMA 等大模型生成文本（如 HC3、OpenLLMText、DeepfakeTextDetect），也有一些数据集采用多模型

混合生成（如 M4、DetectRL），用于模拟真实环境中的多源 AI 文本。在应用场景上，现有数据集覆盖问答与知识类（SQuAD、ELI5）、新闻写作（GROVER）、社交媒体文本（TweepFake）以及学术写作（CHEAT、ArguGPT）等不同领域。本文使用比赛方提供的数据，涉及 GPT、LLaMA、Mistral 三个大模型，同时覆盖书籍、论文、新闻、问答、社媒和评论六个不同文本类型。

1.2 作业概述

本报告将二分类任务进一步发展为多分类任务，即不仅判断输入文本是人类撰写还是由模型生成，还要判断是 GPT、LLaMA 还是 Mistral 模型生成的。报告中的实验分为两个大实验，第一个大实验主要应用“统计与传统机器学习”方法的思路，构建了十维统计特征向量，由于此时我们考虑四分类问题，与之前使用机器学习方法的文献不同，我们之后进行 MLP 得到输出 0、1、2、3。

第二个大实验从另一个角度出发，由于现有文献已经对二分类任务进行了广泛的讨论，那么我们的任务就可以集中于“给定该文本是 AI 生成的，那它究竟是哪一个模型产生的”。再加上相比与学术论文，我们报告中使用的数据集样本较小，我们的第二个实验则基于“零样本与概率统计”方法，实验 2A 主要对 DetectGPT(Mitchell et al., 2023) 进行了复现，使用模型 T5 对文本进行扰动，从而观察那些可以反映扰动带来的变化差异的指标。而实验 2B 研究“给定该文本是 AI 生成的，那它究竟是哪一个模型产生的”这个问题，由于 LLMs 的原理都是最大化概率，因此如果继续使用“零样本与概率统计”方法的思想，直觉上应该效果不好。本文在实验 2B 将“水印”方法的思路进行变相利用，即如果每个大语言模型由于模型结构、训练数据等不同导致不同 LLM 对不同的一组词有生成的偏好，也就是相当于每一个 LLM 有一组 token 它们被采样的概率比其他 token 高，在本文中为致敬水印技术，把这些组 token 称为“Greenlist”。我们利用统计方法恢复各模型的“隐式 Greenlist”，并据此识别文本来源。

总结来讲，实验 1 从“统计与传统机器学习方法”演变，实验 2A 基于“零样本与概率统计方法”，实验 2B 灵感来源于“水印方法”。即除了问题背景中所提的“神经网络方法”，本文将其他三种主流方法均实现了，同时这也为我们横向比较它们潜力的机会。

本文的结构如下：本节作为该项目的介绍；第二节探讨各实验的方法的原理；第三节阐述各方法在比赛方提供的数据集上的表现，并分析了这些表现的可能原因；第四节阐述未来改进方向；第五节是本文的参考文献。

1.3 实验环境

本实验采用 Google Colab 网页版（建议换成终端以防网页连接不稳定...）作为主要计算平台，使用 A100 GPU 进行模型训练与推理加速。运行环境的系统资源配置如下：系统内存为 83.5 GB（当前使用约 4.7 GB），GPU 显存为 40.0 GB（当前使用约 18.0 GB），磁盘空间为 235.7 GB（当前使用约 50.7 GB）。操作系统为 Windows。

实验编程语言为 Python 3，主要依赖 PyTorch 深度学习框架进行模型构建与训练，同时

结合 NumPy、Pandas 进行数据处理，并使用 Scikit-learn 完成数据划分与模型评估。在评估阶段，采用 accuracy、precision、recall、F1-score 以及 ROC-AUC 等指标对模型性能进行全面分析。实验过程中使用 tqdm 进行训练进度可视化，并结合 Python 标准库（如 re、random）完成文本预处理与随机性控制。

1.4 实验代码

实验 1: <https://colab.research.google.com/drive/1GMgoziK1pruFhtN04vHr7IMWzEOap9-g?usp=sharing>

实验 2A: https://colab.research.google.com/drive/11_NeK64NKXo7p75ysbPQmtWOLBiI6jDZ?usp=sharing

实验 2B: https://colab.research.google.com/drive/1xMcSFYK_y1i55FeHQolqu8kvFYVun0ZD?usp=sharing

数据集: <https://drive.google.com/drive/folders/1uhUck6hncp5dZoDHs0daMgafGywKnKSK?usp=sharing>

2 方法

2.1 实验 1

实验 1 的思想基于“统计与传统机器学习方法”思想，构建了十维统计特征向量，然后进行 MLP 得到输出 0、1、2、3。这十维统计特征向量分别是关于 token 在当前上下文下对数概率 $l_t = \log p_\theta(x_t|x_{<t})$ 的五个基础统计量、困惑度特征、两个 Log-rank 特征、概率归一化熵、尾部不确定性。具体计算公式和各自代表含义见下表。

表 1: 实验 1 中提取的文本特征

特征类别	特征名称	计算公式	含义
基础统计量	Mean log-prob	$\mu = \frac{1}{T} \sum_{t=1}^T l_t$	衡量整体平均合理性
基础统计量	Variance	$\sigma^2 = \frac{1}{T} \sum_{t=1}^T (l_t - \mu)^2$	衡量 token 置信度波动程度
基础统计量	Min	$\min(l_t)$	最不确定 token 的对数概率
基础统计量	Max	$\max(l_t)$	最确定 token 的对数概率
基础统计量	Median	$\text{median}(l_t)$	对极端 token 不敏感的中位数
困惑度特征	Perplexity	$\text{PPL}(x) = \exp\left(-\frac{1}{T} \sum_{t=1}^T l_t\right)$	序列整体生成难度
Log-rank 特征	Log-rank	$r_t = \text{rank}(-l_t)$, $\text{LogRank}(x) = \frac{1}{T} \sum_{t=1}^T \log(1 + r_t)$	token 困难程度排序
Log-rank 特征	Rank variance	$r_t = \text{rank}(-l_t)$, $\text{Var}(r_t)$	token 困难程度分布是否均匀
概率归一化熵	Entropy	$H(x) = -\sum_{t=1}^T p_t \log p_t$, $p_t = \frac{e^{l_t}}{\sum_{j=1}^T e^{l_j}}$	token 分布复杂度
尾部不确定性	Tail entropy	$H_{\text{tail}}(x) = -\sum_{t \in \text{bottom } 20\%} p_t \log p_t$	最低置信 token 区域的不确定性

实验中使用 GPT2、GPT2-large 作为模型计算 token 在当前上下文下对数概率 $l_t = \log p_\theta(x_t|x_{<t})$ ，并在计算对数概率时使用 padding、在 MLP 中使用 batch size=64 或 128 加快计算速度。MLP 设定的结构为输入层 10 维，对应前文提取的 10 个统计特征；隐藏层 1，64 个神经元，采用 ReLU 激活函数，并加入 Dropout 0.2 防止过拟合；隐藏层 2，128 个神经

元，同样采用 ReLU 激活函数和 Dropout 0.2。输出层，4 个神经元，对应四个类别 Human、GPT、Llama、Mistral，训练时采用 Softmax 结合交叉熵损失函数进行优化。

2.2 实验 2A

实验 2A 的思想基于 DetectGPT (Mitchell et al., 2023)，其认为由于 LLM 生成文本的原理，如果一段文本是由语言模型生成的，那么它在该模型概率空间中的位置通常是一个局部极大值点，而人类文本则不具有这种性质。因此，如果将这段文本进行一些扰动，比如句子重写、随机码上一个词然后重新填写等，大模型生成这段扰动后的文本的概率往往比生成原本的那段文本低，但人类生成这段新文本的概率可能会降低，也可能会升高，也可能会不变，因为原本文本在概率空间的位置不是极大值点。将新文本的生成概率取平均，则人类生成新文本概率较生成旧文本概率的变化均值更靠近 0，正负会相互抵消，但 LLM 生成新文本的概率总是比原本文本概率更小，变化均值总为负，这就产生了差异。用数学语言表达，设语言模型为 $p_\theta(x)$ ，输入文本为 x ，则其对数概率为 $\log p_\theta(x)$ 。构造扰动文本 $x' \sim q(x' | x)$ ，比较

$$\Delta(x) = \log p_\theta(x) - \mathbb{E}_{x' \sim q(x' | x)}[\log p_\theta(x')].$$

对应的统计量可以写为

$$S(x) = \log p_\theta(x) - \frac{1}{K} \sum_{k=1}^K \log p_\theta(x'_k),$$

其中 x 代表原始文本， x'_k 代表第 k 个扰动样本， K 为扰动次数。直觉来说，LLM 生成文本的 $S(x)$ 较大，人类文本的 $S(x)$ 通常较小，那么我们算出 $S(x)$ 的一个分界值，就可以区分文本是 LLM 生成的，还是人类生成的，即解决二分类问题。

实验中使用 GPT2 作为模型计算 token 在当前上下文下对数概率 $l_t = \log p_\theta(x_t | x_{<t})$ ，T5 构造扰动文本，使用 batch 加快计算速度。扰动次数分别设置为 4 和 10，对比实验结果。

2.3 实验 2B

实验 2B 将“水印”方法的思路进行变相利用，由于每个大语言模型因为模型结构、训练数据等不同可能导致不同 LLM 对不同的一组词有生成的偏好，即我们假设不同模型具有不同的 token 偏好，从而对已知是 LLM 生成的文本做进一步的模型名称的区分。首先，将训练集按照生成模型划分为 GPT、Llama 和 Mistral 三部分。对于模型 j 生成的文本，统计每个 token i 的真实出现次数：

$$N_i^{(j)} = \sum_{x \in D_j} \sum_t \mathbf{1}(x_t = i),$$

其中 D_j 表示模型 j 生成文本集合； x_t 表示第 t 个 token； $\mathbf{1}(\cdot)$ 为指示函数。得到每个 token 出现次数。

随后，由于不同模型具有不同的 token 偏好的假设，对模型 j 我们引入另一语言模型 k

作为参考模型。对于文本

$$x = (x_1, \dots, x_T),$$

参考模型给出下一 token 的概率分布：

$$p_\theta(v|x_{<t}),$$

其中 v 为词汇表中的任意 token。为了降低计算复杂度和小样本的限制，仅保留概率最大的 Top- K 个 token（实验中 $K = 30$ ）。于是 token i 在模型 k 下的期望出现次数估计为

$$E_i^{(k)} = \sum_{x \in D_j} \sum_t p_\theta(i|x_{<t}),$$

其中仅累计 Top- K 概率。

若某一 token 的真实出现次数明显高于参考模型预测，则说明该 token 更受当前模型偏好。因此采用标准化残差

$$z_i^{(j,k)} = \frac{N_i^{(j)} - E_i^{(k)}}{\sqrt{E_i^{(k)}}},$$

其中， $N_i^{(j)}$ ：模型 j 文本中的真实频数； $E_i^{(k)}$ ：参考模型预测的期望频数。该统计量实际上近似服从泊松统计中的标准化残差形式，则 $z > 0$ ：token 比参考模型预测出现更多； $z < 0$ ：token 比预测更少； z 越大，说明 token 越具有模型特征。因此，按照 z_i 由大到小排序，并选取前 Top- K 个 token，即得到各个模型的显示偏好下的 Greenlist：

$$G_j = \{g_1, g_2, \dots, g_K\}.$$

得到三个 Greenlist 之后，为了分析不同模型 token 偏好的相似程度，本实验采用 Jaccard 相似系数衡量任意两个 Greenlist 的重叠程度：

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|},$$

其中 A, B 为两个模型恢复得到的 Greenlist。Jaccard 系数越大，说明两个模型共享的偏好 token 越多；反之，则说明模型具有更明显的区分特征。分别计算 GPT-Llama、GPT-Mistral 以及 Llama-Mistral 的 Greenlist 重叠率。

仅知道 Greenlist 中有哪些 token 仍不足以完成分类，因此进一步利用 z_i 作为 token 权重。对于待检测文本 x ，定义属于模型 j 的 Greenlist 得分：

$$S_j(x) = \sum_{g \in G_j} z_g f(g, x),$$

其中 $f(g, x)$ 表示 token g 在文本中的出现次数， z_g 为对应 Greenlist 权重。最后分别计算

S_{GPT} , S_{Llama} , S_{Mistral} , 并采用最大得分原则完成模型预测,

$$\hat{y} = \arg \max_j S_j(x).$$

3 实验结果

3.1 实验 1

3.1.1 训练集和验证集上表现

表 2: MLP 模型最后 6 个 Epoch 的训练结果

Epoch	Train Loss	Validation Accuracy	Validation F1
95	1.1950	0.5095	0.1688
96	1.1917	0.5095	0.1688
97	1.1911	0.5100	0.1689
98	1.1927	0.5100	0.1689
99	1.1934	0.5100	0.1689
100	1.1908	0.5100	0.1689

以 GPT2-large 作为模型计算 token 在当前上下文下对数概率为例, GPT2 结果相似。从训练过程 (表 2) 可以看出, 模型在约 80 个 epoch 后训练损失趋于稳定, 最终训练损失约为 1.19, 而验证集准确率稳定在 51% 左右, Macro-F1 约为 0.169。此后继续增加训练轮数, 模型性能几乎不再提升, 说明 MLP 已经基本收敛, 即性能瓶颈并非来自模型训练不足。而是说明这十个基本特征能提供的差异信息很小, 即使完美调整 MLP 的参数, 也无济于事。

3.1.2 测试集上表现

我们将 GPT-2 替换为参数规模更大的 GPT-2-large 重新提取特征, 其实也是为了进一步验证模型能力是否构成性能瓶颈。事实也证明, 使用参数更大的模型, 判断准确率没有明显的提升, 甚至有略微下降, 从 53.6% 到 53.5%。因此, 本文认为这个方法在进行我们多分类任务的前景并不明朗。

3.2 实验 2A

3.2.1 设置扰动次数为 4

通过训练集确定最佳分类阈值为 8.266, 对应的假阳性率 (FPR) 为 5%、真阳性率 (TPR) 仅为 16.03%, 在所有真实的人类文本中, 仅有 5% 会被错误地判定为 AI 生成文本; 但在所有 AI 生成文本中, 仅有 16.03% 能够被正确识别出来。最终, 在测试集上的分类准确率为

52.7%，这个二分类的任务上 DetectGPT 和实验 1 使用的 DetectGPT 加上一个 MLP 的效果差不多，略差一点。

是什么原因导致二分类的效果不如人意？我们画出人类文本与 AI 文本对应的 DetectGPT 得分的分布，如图（图一），人类文本与 AI 文本对应的 DetectGPT 得分分布存在较大的重叠区域，两类样本并没有形成明显的分离，仅在人类文本一侧出现了少量长尾分布。这说明经过扰动后，AI 文本与人类文本的对数概率变化虽然存在一定差异，但其判别能力仍然较为有限，因此整体分类性能受到一定限制。这有可能是扰动次数设置太低造成区分不大，因此下面我们设置扰动次数为 10。

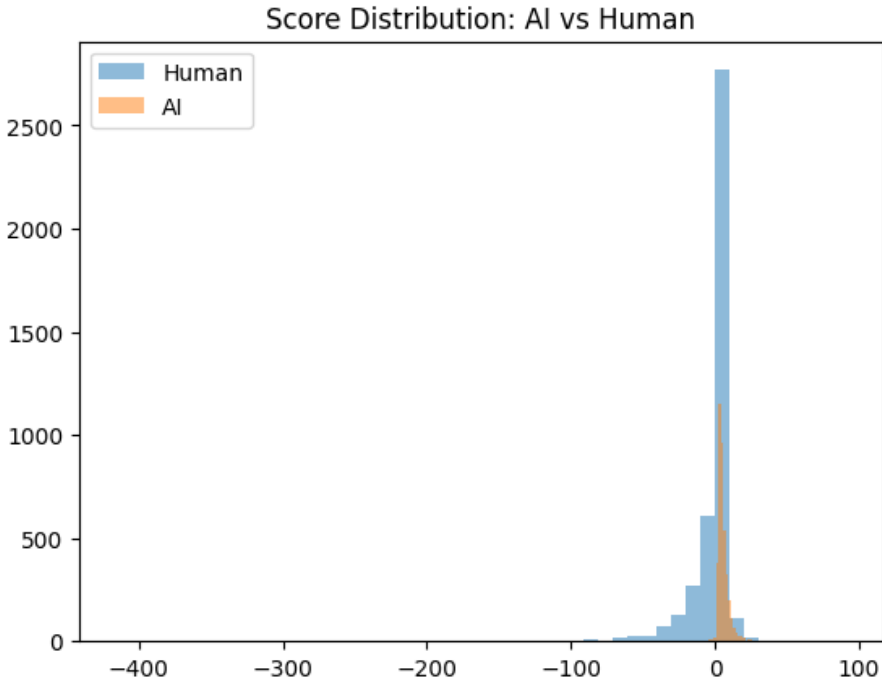


图 1: 人类文本与 AI 文本对应的 DetectGPT 得分分布（扰动次数为 4）

3.2.2 设置扰动次数为 10

将扰动次数由 4 增加至 10 后，重新利用训练集确定分类阈值，得到最佳阈值为 5.1413，对应验证集假阳性率（FPR）为 4.49%，真阳性率（TPR）提高至 21.64%。在测试集上，检测准确率由 52.7% 提升至 53.7%，此时二分类任务结果准确率已经和实验一使用 MLP 结果在同一水平上。与实验 2A 相比，增加扰动次数后，TPR 由 16.03% 提高至 21.64%，说明更多的扰动样本能够更稳定地估计原始文本附近的概率曲率，从而降低了 DetectGPT 统计量的随机波动，使部分原本难以区分的 AI 文本能够被正确识别。同时，FPR 仍维持在约 5% 的水平，表明增加扰动次数并未明显增加对人类文本的误判。

从新的人类文本与 AI 文本对应的 DetectGPT 得分分布可以看到，相较于扰动次数为 4 时，AI 文本整体向较大的 DetectGPT 得分方向移动，而人类文本仍主要集中在较低得分区域，但两方很大一部分样本仍然存在分数相似、无法区分的问题，通过测试集和验证集的真阳性率（TPR）和测试集准确率提升（表 3）也可以看出。因此，DetectGPT 这个方法的有效

性从我们的实验来看似乎并不稳定，现有文献还有提升的空间。

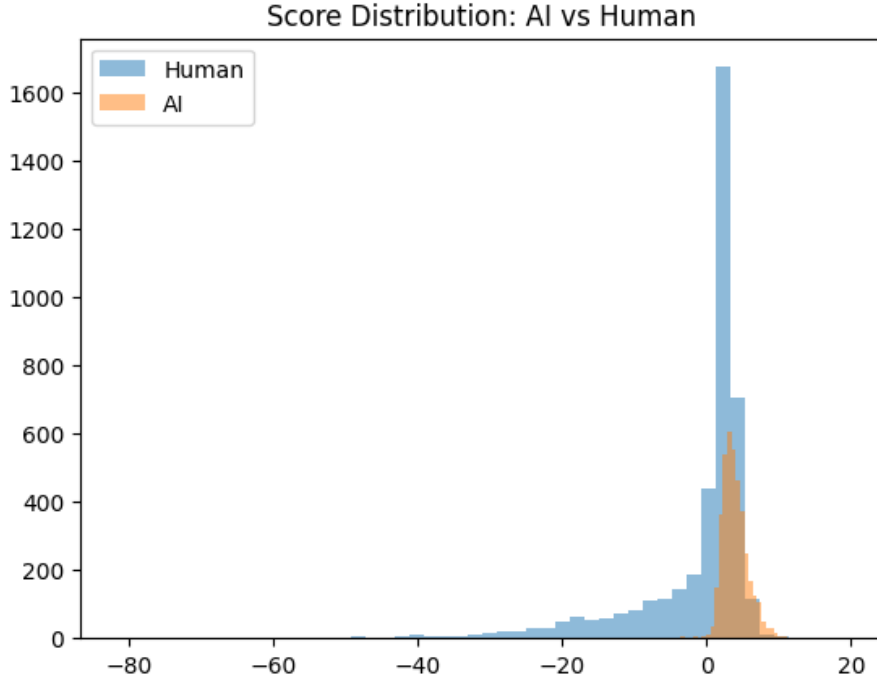


图 2: 人类文本与 AI 文本对应的 DetectGPT 得分分布 (扰动次数为 10)

表 3: 验证集上的表现 (扰动次数为 10)

Class	Precision	Recall	F1-score	Support
0	0.5499	0.9392	0.6937	1020
1	0.7597	0.2000	0.3166	980
Accuracy	—		0.5770	2000
Macro Avg	0.6548	0.5696	0.5052	2000
Weighted Avg	0.6527	0.5770	0.5089	2000

3.3 实验 2B

3.3.1 不同模型 Greenlist 之间的重叠程度

实验 2B 首先利用训练集恢复 GPT、Llama 和 Mistral 三个模型各自的 Candidate Greenlist, 并采用 Jaccard 相似系数衡量不同模型 Greenlist 之间的重叠程度。实验结果表明, GPT 与 Mistral 之间的重叠度最高, 为 0.113; Llama 与 Mistral 之间为 0.075; GPT 与 Llama 之间最低, 仅为 0.037。总体来看, 三组模型之间的 Jaccard 系数均低于 0.12, 说明不同模型恢复得到的高偏好 token 集合具有较低的重叠率, 大部分 token 仅在对应该模型中表现出较高的统计显著性。当然我们的结论需要在更大的数据集上进行验证。通过比赛方提供数据集训练出的 Greelist 见表 4、5、6。

表 4: GPT Greenlist 排名前十

Token ID	Token	N	E	Z-score
28878	...	323	51.501	37.832
1101	...	336	100.773	23.432
28745	;	501	214.857	19.521
5923	.\	395	155.734	19.173
1096	because	518	246.788	17.264
28705	(space)	3410	2557.239	16.863
1024	after	612	320.095	16.316
330	a	455	216.356	16.224
325	(	1583	1056.797	16.187
1040	—	142	40.191	16.059

表 5: Mistral Greenlist 排名前十

Token ID	Token	N	E	Z-score
4100	important	927	428.503	24.082
5806	while	451	195.053	18.326
19814	additionally	466	222.902	16.283
6716	one	322	144.290	14.794
8228	grade	100	33.926	11.344
4658	believe	534	335.125	10.864
512	in	672	444.493	10.791
4477	follows	107	40.038	10.582
3112	it	137	59.860	9.970
1667	main	137	59.865	9.969

表 6: LLaMA Greenlist 排名前十

Token ID	Token	N	E	Z-score
4525	like	651	134.954	44.422
2208	super	312	47.406	38.429
818	in	493	146.643	28.602
1722	as	333	79.370	28.469
1537	but	375	104.036	26.566
3404	stuff	378	105.365	26.560
1119	they	479	180.489	22.220
460	can	2629	1717.738	21.987
0	!	1194	663.980	20.569
11	,	22162	19400.680	19.825

3.3.2 验证集和测试集上的表现

如果在所有样本上直接判断，验证集准确率仅为 14.7%，甚至低于随机猜测，四分类随机准确率约为 25%。从分类报告（表 7）可以进一步发现，模型几乎将所有文本预测为同一类别，仅类别 2 具有较高的召回率，而其他三个类别的召回率接近于 0。这说明，仅依赖现在生成的 Greenlist 中字典 token 的加权匹配，无法充分刻画不同模型之间复杂的生成差异。

造成这一现象的原因本文认为主要有两方面。首先，虽然不同模型恢复得到的 Greenlist 重叠较小，但这些 token 仅代表模型偏好的少量局部词汇，并不能覆盖文本整体的生成特征。当待检测文本较短或 Greenlist token 出现次数较少时，加权得分容易受到少数 token 的影响，从而导致分类结果极不稳定。其次，如果测试集的文本受到了扰动，那么预测会发生特别大的变化。因此，Greenlist 具有较好的模型可解释性，但需要在更大的数据集、文本扰动下确保稳定性。本文认为这是一个比较有潜力的未来研究方向。

表 7: Classification Report on the Validation Set

Class	Precision	Recall	F1-score	Support
0	0.00	0.00	0.00	1020
1	0.24	0.02	0.04	360
2	0.15	1.00	0.25	286
3	0.00	0.00	0.00	334
Accuracy	—		0.15	2000
Macro Avg	0.10	0.26	0.07	2000
Weighted Avg	0.06	0.15	0.04	2000

3.3.3 将实验 2A 和实验 2B 结合进行分类

尽管 Greenlist 直接用于四分类的效果较差，但将 DetectGPT 的结果作为辅助信息后，即给定哪些是 LLM 生成的文本，模型性能得到一些改善。当 DetectGPT 采用 4 次扰动时，引入 Greenlist 后测试准确率为 53.2%；采用 10 次扰动时，测试准确率为 54.9%。

当然，DetectGPT 和 Greenlist 两个方法都具有很大提升空间，这样才能得到更令人满意的测试准确率。

4 未来改进方向

4.1 更多的 LLMs 怎么区分？

本文仅对 GPT、Llama 和 Mistral 三类模型进行了实验。然而，随着大语言模型数量不断增加，直接进行多分类的难度将进一步增加。我们认为有两种解决方法，一是，根据模型背后的框架、原理进行分类，对大类别进行甄别，在一个时间点流行的主流模型框架并没有很多；二是，由于实际上我们做的是识别各个模型生成行为的显示偏好，与微观经济学的显

示偏好理论有一定相似性，借鉴 Arrow (1959) 关于显示偏好的可理性化理论，我们只需要探究模型二元集、三元集之间的显示偏好，就可以建立不同模型偏好之间的一致性与可识别性。本文认为 Greenlist 还是具有一定潜力的。

4.2 能否把 DetectGPT 和 Greenlist 两个方法统一起来？即使用 Greenlist 判断人类？

本文认为是可以的，同样让大语言模型对人类文本的 token 进行预测，使用统计方法识别出一套词组，不过此时不能称为 Greenlist，而是人类的 Blacklist。

4.3 如何得到在更大的数据集、文本扰动下确保稳定性的 Greenlist？

首先，我们需要在更大的数据集上训练，如问题背景提到的数据集。具体来说，对于不同类型的文本，可以确认一个各 LLM 的 Greenlist；另外，比赛方提供的文本数据存在不同的语言，如果各语言对应的数据集足够大，我们应该分别确立各 LLM 的 Greenlist。然后，对于文本扰动，或许可以使用模型 T5 对文本进行至少十次扰动之后，再进行我们的步骤。

4.4 能否进一步提升测试集准确率？

除了上述对模型的提升，还有换现在更主流的模型，GPT2 较老。

参考文献

- [1] Arrow K J. Rational Choice Functions and Orderings[J]. *Economica*, 1959, 26(102): 121–127.
- [2] Beresneva D. Computer-Generated Text Detection: Literature Review[C]. Proceedings of the Student Research Workshop at EACL, 2016: 72–80.
- [3] Jawahar G, Abdul-Mageed M, Lakshmanan L V S. Automatic Detection of Machine Generated Text: A Critical Survey[C]. Proceedings of COLING, 2020: 2296–2309.
- [4] Uchendu A, Le T, Shu K, et al. TuringBench: A Benchmark Environment for Turing Test in the Age of Neural Text Generation[C]. Findings of EMNLP, 2021: 2001–2016.
- [5] Liu Y, Chen Y, et al. ArguGPT: Evaluating, Understanding and Detecting Arguments Generated by Large Language Models[C]. Findings of EMNLP, 2023.
- [6] Li Y, Wang X, et al. DeepfakeTextDetect: Deepfake Text Detection in the Wild[C]. Proceedings of ACL, 2023.

- [7] Wu X, Zhong Y, et al. A Survey on LLM-Generated Text Detection[J]. *Computational Linguistics*, 2024.